

CSCI 135

Final Exam Review

Software Analysis & Design I | Hunter College, Summer 2026

Compiled from the full-semester topic list and student review-quiz feedback

Final Review Session · 7/16/2026 Final Exam · 7/21/2026, 9:50–12:10

COMMON PITFALL

== (Compare) vs. = (Assign)

`==` compares left vs. right and returns true/false without changing either side.

`=` runs right-to-left: evaluate the right side, store it in the left variable, and the whole expression takes that value.

```
int i = 2;
if (i = 3)    // NOT i == 3
{
    cout << "i is " << i;
}
// i is reassigned to 3.
// 3 is non-zero -> true -> body runs
// prints: "i is 3"
// Compiles fine in C++ -- but it's
// almost never what you meant!
```

Rule of Thumb

- In a condition, you almost always want `==`
- 0 is false, any non-zero int is true in C/C++
- Strings cannot be used as a condition —
if (`str = greeting`) simply will not compile
- Some style guides flip the operands
(`3 == i`) so a typo'd `=` triggers a compiler error instead of a silent bug

Method Overload

Same name, similar purpose, different signature — the parameter list differs in number, type, or order. Return type alone is not enough to distinguish two overloads.

```
void display(int num);           // overload 1
void display(const std::string& str); // overload 2
// The compiler picks the right one based on the argument's type at the call site.
```

Overload vs. Override — don't mix these up

Overload: multiple functions, same name, different parameter lists — resolved at compile time.

Override: a subclass redefines a method inherited from its superclass. Name, return type, and parameter list must all match exactly.

Pass by Value vs. Reference vs. Pointer

By Value

a duplicate copy

```
void increase(int num) {  
    num++;  
}  
  
int a = 3;  
increase(a);  
// a is still 3
```

By Reference

the authentic original

```
void increase2(int& num) {  
    num++;  
}  
  
int b = 3;  
increase2(b);  
// b is now 4
```

By Pointer

the original's address

```
void increase(int* p) {  
    (*p)++;  
}  
  
int c = 3;  
increase(&c);  
// c is now 4
```

A pointer parameter is technically pass-by-value too — **it's the address that gets copied**, not the pointed-to data. That's why `(*p)++` reaches through the copy to modify the original int.

Two Common Mistakes

1. Printing instead of returning

```
bool isPrime(int num) {  
    if (num <= 1) {  
        cout << num << " not prime";  
        return false;  
    }  
    ...  
}  
  
// A function should RETURN a value for  
// the caller to use -- not print it itself.
```

2. Missing a return in every path

```
double getMPG() {  
    if (numGallons != 0) {  
        return (double)numMiles  
            / numGallons;  
    }  
    // what if numGallons == 0 ?  
    // no return here -> undefined!  
}
```

Checklist before you submit a function

- ☐ Every branch of every if/else returns a value of the declared type
- ☐ Results are returned, not printed, unless the function's job is I/O
- ☐ {} braces are used whenever a block has more than one statement

Arrays — Fundamentals

7	3	9	1	5
0	1	2	3	4

arr[2] = 9 (index starts at 0)

Know how to...

- Find the number of elements (size passed separately, or `.size()` for `std::array/vector`)
- Access / update a single element by index
- Compute sum, average, max, min, and search for a value with a loop

```
int sum = 0, maxVal = arr[0];
for (int i = 0; i < size; i++) {
    sum += arr[i];
    if (arr[i] > maxVal) maxVal = arr[i];
}
double avg = 1.0 * sum / size;    // 1.0 * avoids integer division
```

Strings — Manipulation

`.length()` / `.size()`

number of characters

`.substr(start, len)`

extract a substring

`.at(i)` or `[i]`

access one character

`+` or `+=`

concatenate strings

`.find(target)`

index of a substring, or npos

`std::to_string(n)`

convert a number to a string

```
std::string str("Hello");  
std::cout << str.length();           // 5  
std::cout << str.substr(2, 3);        // "llo" -- start at index 2, take 3 chars  
// A pointer to a class type uses -> ; a plain object uses . -- see Pointers section
```

Vectors — A Smart, Resizable Array

A vector behaves like an array with built-in operations, and it can grow or shrink at runtime.

Define & initialize

```
std::vector<int> v = {1, 2, 3};
```

.size()

```
number of elements currently stored
```

.push_back(x)

```
append x to the end
```

.pop_back()

```
remove the last element
```

$v[i]$ accesses by index, same as an array — but `push_back` / `pop_back` are what a plain array can't do.

2D Arrays

		grid[1][2]	

Row-major access

```
int grid[3][4];
for (int r = 0; r < 3; r++) {
    for (int c = 0; c < 4; c++) {
        grid[r][c] = r * 4 + c;
    }
}
```

- First index = row, second index = column — `grid[row][col]`
- Nested loops: outer loop walks rows, inner loop walks columns
- A 2D array is really an array of arrays in memory — same rules for bounds and initialization apply per row



MOST REQUESTED TOPIC IN REVIEW-QUIZ FEEDBACK

Pointers & Dynamic Memory

Fundamentals · Initialization · Arrow vs. Dot · Arithmetic · new/delete · Pointers to Objects

Pointer Fundamentals

A pointer stores the address of a variable. The dereference operator `*` gets you back to the value living at that address.



`*ptr` is an alias for `i` -- reading or writing `*ptr` affects `i` directly

```
int i;           // i is not initialized
int* ptr = &i;   // ptr now holds the address of i
*ptr = 5;        // i is now 5
i++;            // i is 6, and *ptr reads as 6 too -- same memory
```

Four Ways to Initialize a Pointer

1. nullptr

```
std::string* p = nullptr;
```

Cannot be dereferenced. *p on a null pointer crashes.

2. Address of an existing variable

```
Type* ptr = &variable;
```

*ptr becomes an alias of variable. No delete needed.

3. new — single element

```
std::string* p = new std::string("Hi");
```

Dynamically allocated. Pair with delete p; then p = nullptr;

4. new[] — array

```
std::string* p = new std::string[n];
```

Dynamically allocated array. Pair with delete[] p; -- the [] cannot be omitted.

Arrow Operator -> vs. Dot Operator .

. Dot

Use when you have an object name directly.

```
std::string str("Hello");  
std::cout << str.length();  
// 5
```

-> Arrow

Use when you have a pointer to a class-type object.

```
std::string* p = &str;  
std::cout << p->length();  
// same as (*p).length()
```

p->length() is shorthand for (*p).length()

The parentheses around *p can't be dropped when written with the dot: the dot operator binds tighter than *, so *p.length() would try (incorrectly) to dereference the result of p.length().

Arrays & Pointer Arithmetic



```
int* arr;           // same idea as: int arr[];
*(arr + 1)          // same as arr[1]
&arr[1]             // same as (arr + 1) -- the address of the 2nd element
*arr + 1            // NOT the same! This is (*arr) + 1, i.e. arr[0] + 1
```

Subarray trick (used constantly in recursion): if you pass `arr + 1` to the next call, you must also pass `size - 1`, or you'll walk right off the end of the array's memory.

Dynamic Memory — new / delete

```
// single element
int* p = new int;
*p = 5;
...
delete p;
p = nullptr;    // avoid dangling ptr
```

```
// array of elements
int* p = new int[10];
...

delete[] p;    // [] required!
p = nullptr;
```

Rules that get tested

- Every new must be paired with exactly one delete
- A single new int needs delete p; — using delete[] p; here is a mismatch
- new int[10] needs delete[] p; — omitting the [] only releases the first element and leaks the rest
- Set the pointer to nullptr right after delete — using a pointer after it's deleted (a "dangling pointer") is undefined behavior

Pointers to Objects

Everything from the last few slides still applies once the pointed-to type is a class instead of int or string.

```
class Account {
public:
    double getBalance() const { return balance; }
    void deposit(double amt) { balance += amt; }
private:
    double balance = 0;
};

Account* acctPtr = new Account();           // dynamically allocated object
acctPtr->deposit(100);                       // -> since acctPtr is a pointer
std::cout << acctPtr->getBalance();          // (*acctPtr).getBalance() also works
delete acctPtr;                             // release it when done
```

A class can also hold a pointer as a data member (e.g. a linked node's next pointer). The same new/delete and nullptr-safety rules apply to that member.

Encapsulation

A class bundles data (private, protected) with the functions that operate on it (public). Data members are reached directly by other methods in the same class — no parameter needed.

```
class Rectangle {  
public:  
    void setWidth(double w) { width = w; }      // setter  
    double getWidth() const { return width; }    // getter  
    double area() const { return width * height; }  
  
private:  
    double width  = 0;  
    double height = 0;  
};  
// width and height are hidden from outside code --  
// only reachable through the public interface above.
```

Constructors & Member Functions

Constructors

- Same name as the class — always
- No return type — not even void
- Runs automatically when an object is created

```
class Point {  
public:  
    Point() : x(0), y(0) {}  
    Point(int a, int b) : x(a), y(b) {}  
private:  
    int x, y;  
};  
  
Point p1;           // calls Point()  
Point p2(3, 4);     // calls Point(int,int)
```

Creating Objects

1. Pick the constructor that matches your arguments
2. The object exists on the stack (or heap, if made with new)
3. Call methods with . for a plain object, -> for a pointer to one

```
double dist =  
    std::sqrt(p1.getX()*p1.getX()  
              + p1.getY()*p1.getY());  
  
Point* pp = new Point(1, 2);  
std::cout << pp->getX();  
delete pp;
```

Enumeration Types

An enum defines a type whose values are a named list of ints — handy as readable case labels in a switch statement.

```
enum Direction {  
    NORTH, SOUTH, EAST, WEST  
};  
// no values given -> starts at 0,  
// increases by 1: N=0 S=1 E=2 W=3
```

```
enum MyEnum {  
    ONE = 7, TWO, THREE,  
    FOUR = -3, FIVE  
};  
// ONE=7 TWO=8 THREE=9 FOUR=-3 FIVE=-2
```

- Unset identifiers take the previous value + 1; the very first defaults to 0
- Two or more names in the same enum can share the same int value
- Treat an enum as its own distinct type — use it for labels, and avoid doing arithmetic on enum variables even though the values are backed by int

BUILT FOR MORE WORKED EXAMPLES — REQUESTED IN FEEDBACK

Recursion

The big idea · base-case safety · array sum & max · palindrome · reversal · digit counting

The Big Idea

A function that calls itself, directly or indirectly, to solve smaller and smaller versions of the same problem.

Base Case

The simplest version of the problem — answered directly, with no further recursive call. This is what stops the recursion.

Recursive Case

Break the problem into one small piece plus a smaller version of the same problem, then combine the results.

The Recursive Leap of Faith

Don't trace the whole call chain in your head. **Trust** that your function already works correctly on a smaller input. Your only job when writing the recursive case is figuring out how to combine that trusted smaller result with the one piece you're looking at right now.

Example: $\text{Total Sum} = (\text{First Element}) + (\text{Sum of the Rest})$

Base-Case Safety & the Pointer Trick

Base-Case Safety

- Arrays / strings: what happens when `size == 0` or `size == 1`?
- Integers: what happens when the number is a single digit, i.e. in `[-9, 9]`?
- If your function crashes or infinite-loops, it's almost always because the base case never gets reached.

Shrinking an Array Recursively

`arr + 1` moves the starting pointer to the next element. **Whenever you pass `arr + 1` to the next call, pass `size - 1` too** — otherwise you'll walk off the end of the array.

```
// Thinking about integers, digit by digit:
int lastDigit = num % 10;    // rightmost digit
int rest      = num / 10;    // everything EXCEPT the rightmost digit
// Prefer peeling off the rightmost digit -- it's the clean, reliable direction for recursion.
```

RECURSION

Worked Example: Array Sum & Max

Sum: {1, 2, 3} -> 6

```
int arraySum(int arr[], int size) {  
    if (size == 0) {           // base case  
        return 0;  
    }  
    return arr[0]  
        + arraySum(arr + 1, size - 1);  
    //    ^first      ^rest of array  
}
```

Max: {1, 2, 3} -> 3

```
int arrayMax(int arr[], int size) {  
    if (size == 1) {           // base case  
        return arr[0];  
    }  
    int restMax = arrayMax(arr + 1, size - 1);  
    return (arr[0] > restMax)  
        ? arr[0] : restMax;  
}
```

Same skeleton both times: check the base case first, otherwise combine `arr[0]` with the recursive result on the remaining size - 1 elements.

Worked Example: Palindrome Check

An array is a palindrome if it reads the same forward and backward. {1,2,1} -> true. {1,2,3} -> false.

```
bool isPalindrome(int arr[], int size) {  
    if (size <= 1) {                // base case: 0 or 1 elements  
        return true;  
    }  
    if (arr[0] != arr[size - 1]) {   // ends don't match  
        return false;  
    }  
    // ends match -- check the middle (cut from BOTH the head and the tail)  
    return isPalindrome(arr + 1, size - 2);  
}
```

Cutting from both ends: arr + 1 skips the first element, and size - 2 accounts for removing one from each end — not just one from the front.

RECURSION

Worked Example: Reversing a String

Return a NEW reversed string ("abc" -> "cba")

```
string reverse(string s) {  
    if (s.size() <= 1) {    // base case  
        return s;  
    }  
    return reverse(s.substr(1))  
        + s[0];  
    // reversed rest, then first char last  
}
```

In-place reversal, void return (s becomes "cba")

```
void reverseInPlace(string& s,  
                    int start, int end) {  
    if (start >= end) return;    // base  
    std::swap(s[start], s[end]);  
    reverseInPlace(s, start + 1,  
                  end - 1);  
}
```

Return type string: you're building a new string with +. **Return type void:** you're swapping characters inside the original memory with std::swap.

Worked Example: Counting Digits

countDigits(125) -> 3 countDigits(-15) -> 2

```
int countDigits(int num) {  
    if (num < 0) {  
        return countDigits(-num);    // handle a negative sign once  
    }  
    if (num >= -9 && num <= 9) {      // base case: single digit  
        return 1;  
    }  
    return 1 + countDigits(num / 10); // strip the rightmost digit, count 1 for it  
}
```

num / 10 drops the rightmost digit through integer division — the same "peel off the end, recurse on the rest" pattern as every other example on this slide deck.